

DATA STRUCTURES AND ALGORITHMS ASSIGNMENT

Exercise 2: E-commerce Platform Search Function

1. Understand Asymptotic Notation:

a. Explain Big O notation and how it helps in analyzing algorithms.

Big O notation is a mathematical representation used to describe the time complexity or space complexity of an algorithm in terms of input size (n). It helps analyze how the performance of an algorithm scales as the input grows, allowing us to compare algorithms and choose the most efficient one.

b. Describe the best, average, and worst-case scenarios for search operations.

Best, Average and Worst Case for Search Operations:

Case	Linear Search	Binary Search
Best	$O(1)$ — element found at first position	$O(1)$ — element found at middle
Average	$O(n)$ — element found somewhere in middle	$O(\log n)$ — halves search space each time
Worst	$O(n)$ — element at last position or not found	$O(\log n)$ — element at last division or not found

2. Setup:

Create a class **Product** with attributes for searching, such as **productId**, **productName**, and **category**.

CODE:

Product.java

```
package MOD_2.exercise_2;
```

```
public class Product {  
    private int productId;  
    private String productName, category;
```

```
public Product(int productId, String productName, String category) {  
    this.productId = productId;  
    this.productName = productName;  
    this.category = category;  
}  
  
public int getProductId() {  
    return productId;  
}  
  
public void setProductId(int productId) {  
    this.productId = productId;  
}  
  
public String getProductName() {  
    return productName;  
}  
  
public void setProductName(String productName) {  
    this.productName = productName;  
}  
  
public String getCategory() {  
    return category;  
}  
public void setCategory(String category) {  
    this.category = category;  
}  
}
```

3.Implementation:

- a. Implement linear search and binary search algorithms.
- b. Store products in an array for linear search and a sorted array for binary search.

ProdComparator.java

```
package MOD_2.exercise_2;
```

```
import java.util.*;
```

```
public class ProdComparator implements Comparator<Product> {
```

```
    public int compare(Product p1, Product p2){
        if(p1.getProductld() > p2.getProductld()){
            return p2.getProductld();
        }
        return p1.getProductld();
    }
}
```

Main.java

```
package MOD_2.exercise_2;
```

```
import java.util.*;
```

```
public class Main {
    public static Product linearSearch(Product[] p, int targetld) {
        for (int i = 0; i < p.length; i++) {
            if (p[i].getProductld() == targetld) {
                return p[i];
            }
        }
    }
}
```

```

    }

    return null;
}

public static Product binarySearch(Product[] p, int targetId) {

    int low = 0, high = p.length - 1;

    while (low <= high) {

        int mid = (low + high) / 2;

        if (p[mid].getProductId() == targetId) {

            return p[mid];

        } else if (p[mid].getProductId() < targetId) {

            low = mid + 1;

        } else {

            high = mid - 1;

        }

    }

    return null;
}

```

```

public static void main(String args[]){

    Scanner sc=new Scanner(System.in);

    System.out.println("Enter number of products you want to enter: ");

    int n=sc.nextInt();

    Product p[]=new Product[n];

    for(int i=0;i<n;i++){

        System.out.println("Enter details of product "+(i+1)+" : ");

        System.out.println("Enter product Id : ");
    }
}

```

```

        int id=sc.nextInt();

        System.out.println("Enter product name : ");

        String name=sc.next();

        System.out.println("Enter product category : ");

        String category=sc.next();


        p[i]=new Product(id,name,category);
    }


    System.out.print("\nEnter productId to search: ");
    int targetId = sc.nextInt();


    // Linear Search

    System.out.println("\n--- Linear Search ---");

    Product result1 = linearSearch(p, targetId);

    if (result1 != null) {

        System.out.println("Product found: " + result1.getProductName() + " | Category: " +
result1.getCategory());
    } else {

        System.out.println("Product not found.");
    }


    Arrays.sort(p, new ProdComparator());


    // Binary Search

    System.out.println("\n--- Binary Search ---");

    Product result2 = binarySearch(p, targetId);

    if (result2 != null) {

        System.out.println("Product found: " + result2.getProductName() + " | Category: " +
result2.getCategory());
    }

```

```

    } else {
        System.out.println("Product not found.");
    }
}
}

```

OUTPUT:

```

Enter number of products you want to enter:
3
Enter details of product 1:
Enter product Id :
1
Enter product name :
chips
Enter product category :
snacks
Enter details of product 2:
Enter product Id :
2
Enter product name :
skirt
Enter product category :
garment
Enter details of product 3:
Enter product Id :
3
Enter product name :
smartphone
Enter product category :
electronics

Enter productId to search: 2

--- Linear Search ---
Product found: skirt | Category: garment

--- Binary Search ---
Product found: skirt | Category: garment

```

4. Analysis:

a. Compare the time complexity of linear and binary search algorithms.

Time Complexity Comparison:

Algorithm	Time Complexity	Space Complexity
Linear Search	$O(n)$	$O(1)$
Binary Search	$O(\log n)$	$O(1)$

b. Discuss which algorithm is more suitable for your platform and why.

For an e-commerce platform, binary search is clearly the better choice. E-commerce websites have millions of products. If we use linear search, in the worst case we'd be going through every single product one by one just to find what we're looking for. That is not practical at scale.

Binary search solves this by halving the search space at every step. So even with 1 million products, it finds the result in at most 20 comparisons.

However, binary search needs the array to be sorted beforehand, which is why we sort by productId before searching. This is a one-time overhead that's completely worth it given the performance gains during actual search operations.

Linear search works fine for small datasets or when the data is unsorted and we can't afford the sorting cost. But for a real e-commerce platform, binary search is the way to go.